INTELLIGENT SECURITY FOR CONTAINERS

Project Planning Brief & Implementation Roadmap

ML-Based Runtime and Orchestration Threat Classification Using the Jev Model

Purpose	Give a planning agent (Claude) a consolidated, implementation-oriented brief so it can help turn the design into a working project.
Project setting	Cloud Security and SRE; isolated Kubernetes laboratory; multi-tenant workload simulation.
Core idea	Collect multiple security signals, add workload context, classify suspicious activity, then apply a safe, auditable response policy.
Current status from report	Design-stage proposal. The report says the full pipeline was not implemented, Jev API access had not been obtained, and experimental results were not yet available.

Prepared from the supplied project report and architecture diagram, with implementation recommendations clearly distinguished from the proposed design.

1. Executive Summary

This project is an AI-assisted Kubernetes runtime threat detection and response pipeline. It is not just a model or a dashboard. The intended system observes container images, runtime behaviour, Kubernetes control-plane activity and network flows; normalizes those signals; adds workload-specific baseline context; classifies suspicious events; applies a deterministic policy; and records every decision for evaluation.

The project aims to investigate whether a fast typed-decision model (Jev) can classify runtime and orchestration events when given structured event facts and a workload's expected behaviour. The model is one replaceable component, not the entire security system.

Recommended practical strategy: build a reliable end-to-end minimum viable pipeline using a local Kubernetes cluster, Falco, Python normalization, rules and audit logging first. Add the other telemetry sources incrementally. Keep the Jev integration behind an adapter because the supplied report states that API access was still pending. Evaluate against a rule-based detector and a classical ML baseline before enabling disruptive automated responses.

One-sentence description

Build a system that detects suspicious Kubernetes container activity, uses workload context to classify likely threats, and chooses a safe response—log, alert, human review or quarantine—with an auditable record.

Project status and evidence boundary

- **Source-derived:** the supplied report describes a design-stage system, a proposed telemetry pipeline, Jev typed questions, a decision policy, evaluation metrics and risk mitigations.
- **Not yet established in the report:** that the full system works end to end, that Jev is accessible to the team, that Jev improves accuracy, or that the illustrative thresholds and example probabilities are valid.
- **Planning recommendation:** mark all components as Proposed / In Progress / Implemented / Tested and distinguish measured results from illustrative examples in the final report.

2. Problem Statement

Easy Hinglish version

Ek Kubernetes cluster mein different applications ya tenants ke containers shared infrastructure par run karte hain. Image scan karne se pata chal sakta hai ki image mein known vulnerabilities hain, lekin container start hone ke baad woh kya processes run kar raha hai, kaunse permissions use kar raha hai, kis host resource ko access kar raha hai, ya kis network destination se connect kar raha hai—ye sab image scan akela nahi bata sakta.

Project ka problem hai runtime behaviour, Kubernetes audit events aur network activity ko context ke saath analyse karna, taaki system normal aur suspicious behaviour mein difference kar sake. Suspicious event ko classify karke system alert, human review ya controlled quarantine trigger kare.

Formal problem statement

Most container-security automation focuses on static artifacts such as images. Runtime and orchestration threats—including container escape, privilege abuse and resource hijacking—appear in heterogeneous signals such as system calls, Kubernetes audit records and network flows. Their meaning depends on workload context. This project proposes and evaluates a pipeline that combines those signals with per-workload behavioural baselines, classifies suspicious events using typed model outputs, and applies a confidence-gated, auditable response policy.

Why current approaches are insufficient

Approach / source	What it provides	Gap this project addresses
Trivy / image scanning	Image vulnerability and configuration findings	Does not directly describe what a running container does after launch.
Falco / runtime rules	Runtime events and known suspicious patterns	Rules need maintenance and may lack workload-specific context.
Kubernetes audit logs	API activity, permission changes and control-plane operations	An event's security significance needs context and correlation.
Cilium + Hubble	Workload network-flow visibility	A flow alone may not show whether the activity is malicious.
Classical ML	Numeric feature-based classification or anomaly scoring	Requires feature engineering and suitable representative data.

3. What Is Actually Being Built?

Subsystem	Purpose	Initial implementation idea
Kubernetes lab	Isolated environment with multiple simulated tenants and workloads	kind or Minikube; separate namespaces for tenant-a, tenant-b, tenant-c.
Telemetry collection	Collect events from image, runtime, orchestration and network layers	Trivy, Falco, Kubernetes audit logs, Cilium/Hubble; add incrementally.
Baseline store	Represent expected behaviour per workload	Record expected processes, mounts/privileges, network patterns and resource behaviour.
Normalizer	Convert heterogeneous raw events to one structured schema	Python service; extract deterministic facts in code.
Pre-filter	Reduce unnecessary model calls and latency	Simple rules and thresholds; measure missed-event risk.
Classifier interface	Classify attack type, severity and evidence flags	Adapter interface; rules and scikit-learn baselines first; Jev adapter if access exists.
Decision engine	Apply explicit confidence and severity policy	Python code; separate model output from enforcement.
Response integration	Notify or contain suspicious activity	Logs, alerts, supervised NetworkPolicy quarantine; termination disabled initially.
Audit and evaluation	Make decisions reviewable and compare approaches	Persist event, state, prediction, scores, model/version, action and reviewer label.
Observability	Show system health and security metrics	Prometheus/Grafana; Slack/email alerting as needed.

Out of scope for the first version

- Production deployment across real customer tenants.
- Unsupervised automatic pod termination before safety evaluation.
- A claim that the system detects every class of container attack.
- A claim that Jev is superior before a fair held-out comparison.
- A large, general-purpose commercial security platform.

4. End-to-End Workflow

1. **Collect telemetry.** Trivy provides image-scan findings; Falco provides runtime signals; Kubernetes audit logs provide control-plane/API events; Hubble provides network-flow context.
2. **Normalize and enrich.** Python converts each relevant event to a common schema, joins Kubernetes metadata and adds workload baseline context. Compute facts that code can verify exactly.
3. **Pre-filter.** Use cheap deterministic rules to drop clearly normal events or route known patterns. Measure whether the pre-filter misses any labelled attack events.
4. **Classify.** Send the structured state to a classifier. The proposed Jev path asks for attack type (Choice), severity (Score) and selected evidence flags (Noul).
5. **Apply policy.** The Python decision engine checks the predicted class, confidence, severity, evidence agreement and explicit safety conditions.
6. **Respond.** Log, alert, request human review or quarantine through Kubernetes controls. Termination remains disabled until evaluation and recovery tests justify it.
7. **Audit and improve.** Record all decisions; label outcomes; evaluate metrics; adjust state fields, thresholds, descriptions and model versions using training/tuning data only.

Illustrative event

A payment-service pod unexpectedly starts a shell and has a host runtime socket mounted. Falco emits the runtime event; Kubernetes metadata identifies the workload; the baseline says that workload normally does not start shells; the normalizer records the mounted socket as a deterministic fact; the classifier may predict container_escape; the policy engine checks evidence and confidence; the system alerts and may quarantine under a validated policy; the decision is logged. This is an example, not a measured outcome.

5. Jev Model: Intended Role and Fallback

The supplied report describes Jev as a typed-decision model that receives a state plus pre-defined typed questions and returns constrained outputs rather than free-form text. Treat this description as report-provided/vendor-described information; verify the current API, model access, schema, rate limits, versioning and terms before implementation.

Question type	Question	Expected result	Use
Choice	Which class best matches the event?	One option plus option probabilities	Attack class selection.
Score	How severe is the event?	Ordered severity score/levels and confidence information	Risk ranking and policy thresholds.
Noul	Is a specific evidence claim true?	Probability that the claim is true	Cross-checking the main class and routing uncertain events.

Initial attack classes

- **container_escape:** suspicious access to host resources/runtime, sensitive host paths or privileged container capabilities.
- **privilege_abuse:** misuse of Kubernetes permissions, service-account tokens, secrets or execution into other pods.
- **cryptomining:** unexpected sustained compute use combined with suspicious binaries or outbound network patterns.
- **benign:** normal workload or legitimate administration.
- **unknown:** insufficient or ambiguous evidence; route to review rather than forcing an attack label.

Initial evidence flags

- **baseline_deviation:** behaviour is inconsistent with the workload’s normal pattern.
- **shell_unexpected:** an interactive shell is unusual for this workload.

- **resource_exposure:** the event provides access to host or cluster resources that the workload should not have.
- **resource_abuse:** resource/network behaviour is consistent with unauthorized compute use.

Integration requirement

Define a narrow classifier interface, for example classify(state) → {class, severity, evidence, confidence, model_version}. Do not couple the pipeline to Jev-specific response fields. Keep rules and classical ML implementations behind the same interface. If Jev is unavailable or times out, use a documented safe fallback; do not silently treat missing output as benign.

6. Event State and Data Design

A consistent state format is necessary so that events from different sources can be classified and evaluated consistently. Keep raw evidence separately from normalized fields so investigators can trace a decision back to its source.

Field group	Example fields	Source / rationale
Identity	timestamp, cluster, namespace/tenant, pod, container, image	Kubernetes metadata; identify affected workload.
Process	process name/path, parent process, event/action, relevant system call	Falco/runtime telemetry.
Configuration facts	privileged mode, host path/socket mounts, capabilities, service account	Pod spec and Kubernetes API; deterministic facts where possible.
Orchestration	API verb, resource, identity/subject, role/permission changes, secret access	Kubernetes audit logs.
Network	source/destination workload, destination, protocol, direction, flow metadata	Hubble/Cilium.
Image findings	vulnerability count/severity, misconfiguration findings, image identity	Trivy.
Resource usage	CPU/memory trend and relevant time window	Metrics; useful for resource abuse context.
Baseline	expected processes, mounts, network destinations, expected resource range	Per-workload baseline store.
Classification/audit	predicted class, probabilities/confidence, severity, evidence flags, action, reviewer label	Classifier and decision engine.

Illustrative JSON state

```
{
"event_id": "lab-event-001",
"timestamp": "2026-10-17T10:00:00Z",
"workload": {
"tenant": "tenant-b",
"namespace": "tenant-b",
"pod": "payments-demo",
"container_image": "payments-demo:lab"
},
"runtime_event": {
"process": "/bin/sh",
"parent_process": "nginx",
"source": "falco"
},
"facts": {
"host_runtime_socket_mounted": true,
"privileged_container": false
},
"orchestration": {},
"network": {},
"baseline": {
"shell_expected": false,
"expected_network_profile": "payments-baseline-v1"
}
}
```

This is a suggested schema for planning, not a vendor-required format. Add schema validation, event timestamps, source identifiers and explicit missing/unknown values. Do not turn missing telemetry into a false claim that an activity did not occur.

7. Baselines, Lab Scenarios and Dataset

Laboratory setup

- Create a local cluster using kind or Minikube.
- Create three namespaces to simulate tenants A, B and C.
- Deploy representative web, API, database and payment-style demo workloads.
- Observe legitimate workload behaviour before running suspicious scenarios.
- Use an isolated lab; never run attack simulations against production or third-party systems.

Initial scenario set

Scenario class	Safe lab-level scenario intent	Expected evidence
Benign	Normal deployment, restart, request handling and authorized admin activity	Expected processes, API calls and network patterns.
Container escape indicators	Controlled lab misconfiguration that exposes a sensitive host/runtime resource; validate detection without weaponized exploitation	Sensitive mount/socket facts, unexpected process or access event.
Privilege abuse indicators	Controlled test of an over-permissive role or unexpected service-account access	Audit event, principal, resource and permission context.
Cryptomining/resource abuse indicators	Benign load-generator workload plus simulated suspicious destination metadata; do not deploy real miners	Sustained CPU pattern and unusual outbound-flow evidence.
Unknown/ambiguous	Incomplete, conflicting or insufficient telemetry	System should avoid forced confident classification and route to review.

Labeling and split discipline

- Label each generated event using the known scenario ground truth and analyst review where needed.
- Keep related events from the same scenario/run grouped when splitting data to reduce leakage.
- Create a tuning set and a held-out test set before tuning thresholds or prompts/descriptions.
- Include diverse benign events; benign diversity is essential for meaningful false-positive measurement.
- Document that a small synthetic lab may not generalize to real clusters.

8. Decision Policy and Safe Response

Keep enforcement logic in ordinary code. Model predictions are evidence inputs, not commands. The supplied report gives provisional severity bands and an initial confidence threshold, but those values are illustrative starting points and must be calibrated against data.

Condition	Suggested initial action	Safety notes
Confident benign classification	Log only	Continue monitoring; preserve relevant audit trail.
Unknown, low confidence or conflicting signals	Human review	Do not automatically terminate or mark as safe.
Low-severity suspicious class	Low-priority alert	Collect more evidence if appropriate.

Condition	Suggested initial action	Safety notes
Medium-severity suspicious class	Alert + review	Require corroborating signals for disruptive action.
High severity + validated evidence and policy	Alert + supervised/reversible quarantine	Test NetworkPolicy behaviour, rollback and service impact.
Critical escape evidence, after validation	Consider termination only after staged validation	Disabled in the initial version; require explicit safety guardrails.

Mandatory guardrails

- Use least-privilege credentials for the response controller.
- Restrict automated actions to lab namespaces and explicitly allowlisted workloads.
- Default to alert/review if the classifier is unavailable, times out or returns invalid output.
- Use reversible quarantine where possible and document a rollback procedure.
- Do not delete pods automatically in the MVP.
- Sanitize and delimit attacker-controlled strings before model submission; treat event text as untrusted data.
- Redact secrets and sensitive identifiers before sending telemetry to a hosted model.
- Log every policy decision, action outcome and failure; do not store secrets in logs.

9. Architecture Sketch

The following is a logical view. It is intended to guide implementation; exact deployment boundaries can be adjusted after the MVP works.

```
Telemetry sources
Trivy | Falco | Kubernetes audit | Hubble
|
v
Collector / event adapters
|
v
Normalizer + deterministic fact extraction <--- Baseline store
|
v
Rule-based pre-filter
|
v
Classifier interface (rules / ML / Jev)
|
v
Policy + confidence gate
/ \
v v
Human review Response controller
|
Alert / quarantine
|
v
Audit + metrics
|
v
Evaluation / improvement
```

Do not build all telemetry adapters simultaneously. First complete one vertical slice: one source → normalized event → classifier → decision → audit record.

10. Recommended Implementation Roadmap

Phase	Build	Definition of done
0. Scope and repo	Repository structure, config, schema, threat classes, lab boundaries	A written plan, reproducible setup instructions and an agreed event schema.
1. Kubernetes lab	kind/Minikube, namespaces, demo workloads	Cluster can be recreated and workloads produce normal events.
2. First telemetry source	Falco runtime event integration	A runtime event is ingested and stored with source metadata.
3. Normalizer + audit	Python event schema, deterministic facts, persistent event log	Input event becomes valid normalized JSON and is traceable.
4. Rules + end-to-end MVP	Pre-filter, simple classifier, policy and logging	One benign and one suspicious scenario run end to end.
5. Baseline + scenarios	Workload baseline, labels, benign and suspicious scenarios	Repeatable dataset and explicit ground truth exist.
6. ML comparison	Simple scikit-learn classifier; split discipline	Metrics computed on held-out data.
7. More telemetry	Trivy, Kubernetes audit and Hubble adapters	Signals are normalized and useful to classification.
8. Jev adapter	Verify API/access; implement typed questions and response parsing	Contract tests pass; failure mode is safe; model version logged.
9. Supervised response	Alerts, review workflow, lab-only NetworkPolicy quarantine	Quarantine and rollback tested without unintended scope.
10. Final evaluation/demo	Metrics, dashboard, reproducible demo and limitations	Report separates measured results from assumptions and examples.

Suggested repository structure

```
intelligent-container-security/
README.md
docs/
architecture.md
threat-model.md
evaluation-plan.md
runbook.md
deploy/
kind/ # or minikube instructions
namespaces/
workloads/
falco/
network-policies/
schemas/
event.schema.json
collectors/
falco/
trivy/
kubernetes-audit/
hubble/
services/
normalizer/
baseline/
prefilter/
classifier/
interface.py
rules.py
classical_ml.py
jev_adapter.py
decision_engine/
response_controller/
storage/
migrations-or-schema/
dashboard/
tests/
unit/
integration/
scenarios/
evaluation/
datasets/
scripts/
results/
```

This is a proposed organization, not a requirement from the supplied report. Keep the repository simpler if the MVP does not yet need separate services.

11. Evaluation Plan — What Makes This a Strong Project?

Compare on the same held-out events

Approach	Purpose	Fair comparison requirement
Rule-based detector	Transparent known-pattern baseline	Same labelled event set; document which rules are enabled.
Classical ML	Test feature-based classification, e.g. Random Forest; consider anomaly detection only if labels/data justify it	Train only on training data; tune on tuning/validation data.
Jev typed classifier	Test context-based typed judgments	Use a pinned model version if available; same held-out events; log latency and cost.

Metrics to report

- **Per-class precision, recall and F1:** reveal which threat classes are missed or over-predicted.
- **Macro-F1 and confusion matrix:** prevent a large benign class from hiding poor attack-class performance.
- **False-positive rate on benign events:** important because false alarms and unnecessary quarantine harm reliability.

- **Calibration / confidence:** check whether higher confidence corresponds to better accuracy; do not assume it.
- **Severity agreement:** compare model severity ranking with analyst-assigned severity.
- **End-to-end latency:** median and p95, not just model API time.
- **Cost per 1,000 events:** include model usage and relevant infrastructure costs.
- **Operational metrics:** missed events from pre-filtering, alert volume, response success/failure and recovery time.

Ablation tests

- With vs without workload baseline in the state.
- With vs without pre-filtering.
- With vs without detailed class descriptions.
- Different confidence thresholds and their false-positive / false-negative trade-off.
- Single-source vs multi-source evidence where the dataset supports this comparison.

A meaningful research claim would be something measurable, such as: adding workload baseline context changes macro-F1 or false-positive rate compared with the same classifier without baseline context. Do not promise improvement before measuring it.

12. Risks, Limitations and Mitigations

Risk	Why it matters	Mitigation
Jev access or availability	Hosted API may not be accessible or may be rate-limited.	Keep classifier replaceable; build rules/ML first; safe fallback.
False positives	Healthy service could be isolated or interrupted.	Review-first, reversible quarantine, staged automation and rollback tests.
Missed attacks	Pre-filtering or incomplete sensors may hide relevant signals.	Measure pre-filter recall; preserve raw logs; test multi-source scenarios.
Untrusted event text	Process arguments/image names may be attacker-controlled.	Sanitize, delimit and truncate; rely on deterministic facts for critical checks.
Sensitive telemetry	Identifiers, paths or secrets may be exposed to hosted services.	Minimize and redact fields; review data-handling terms; self-host option if needed.
Small synthetic dataset	Results may not generalize to production.	Include diverse benign workloads; disclose limits; use public data only when compatible.
Model/version drift	Outputs can change after an upgrade.	Pin version, log version, rerun evaluation before upgrading.
Response permissions	Overprivileged controller could damage workloads.	Least privilege, namespace scope, allowlists and policy tests.
No model explanation	A typed output may not explain its decision.	Keep evidence, raw event, normalized state, probabilities, version and action in audit log.
Baseline drift	Normal workloads evolve over time.	Version baselines, review updates, detect drift and avoid automatic unreviewed baseline learning.

13. Key Design Decisions to Resolve Before Coding

1. **What is the MVP threat class set?** Recommended initial set: benign, container escape indicators, privilege abuse indicators, cryptomining/resource abuse indicators, unknown.
2. **What event store will be used?** Start with a simple persistent local store or JSONL for the MVP; choose a database only when query volume and audit needs justify it.

3 **What is the baseline representation?** Define which fields are observed, time windows, how expected values are updated, and how legitimate changes are approved.

4 **What is the canonical event schema?** Version it and validate every source adapter.

5 **How are related events correlated?** Decide how to associate process, pod, API and network events by timestamp, namespace, pod/workload and correlation window.

6 **How does the system fail safely?** Define behaviour for missing sources, stale baselines, invalid classifier responses, timeouts and response-controller errors.

7 **What exactly counts as ground truth?** Use controlled lab scenarios, scenario IDs and reviewed labels; document ambiguous cases.

8 **What actions are enabled?** Begin with log and alert. Enable lab-only quarantine only after tests; leave termination disabled initially.

9 **How will you demonstrate value?** Predefine metrics, hold-out split and baseline comparisons before tuning.

14. Ready-to-Paste Prompt for Claude

Give Claude this brief together with the original project report and architecture diagram if available. The prompt below asks Claude to plan before making large code changes.

You are my senior Cloud Security, Kubernetes, SRE and applied-ML engineering partner. Review this project planning brief and the original project report/architecture diagram carefully. Help me turn the design into a working, reproducible academic project.

PROJECT GOAL
Build an AI-assisted Kubernetes runtime threat detection and response pipeline. Collect telemetry, normalize events, add per-workload behavioural baseline context, classify suspicious activity, apply a safe deterministic policy, and record decisions for evaluation.

IMPORTANT TRUTH CONSTRAINTS
- The original report describes a design-stage proposal, not a completed implementation.
- Jev API access was reported as pending. Do not assume the API exists, is accessible, or performs as claimed. Verify access and current documentation before coding against it.
- Treat all example probabilities, severity thresholds and example outcomes as illustrative until measured.
- Do not invent experimental results, source capabilities, API schemas or successful actions.
- Separate facts from the supplied report, assumptions, and your recommendations.
- If a requirement is unclear, identify it and propose a small test to resolve it.

ARCHITECTURE
Telemetry sources: Trivy, Falco, Kubernetes audit logs, Cilium/Hubble.
Pipeline: collectors -> versioned normalized event schema -> deterministic fact extraction and workload baseline -> rule-based pre-filter -> classifier adapter (rules, classical ML, Jev if accessible) -> deterministic policy/confidence gate -> log/alert/human review/lab-only quarantine -> audit/evaluation.
Initial classes: benign, container_escape indicators, privilege_abuse indicators, cryptomining/resource abuse indicators, unknown.
Jev questions proposed: Choice for attack class, Score for severity, Noul for evidence flags.
Initial evidence flags: baseline_deviation, shell_unexpected, resource_exposure, resource_abuse.
Response must fail safe. Log and alert first; do not enable pod termination in the MVP. Any quarantine must be lab-scoped, reversible, tested and least-privileged.

YOUR TASKS
1. Start with a critical review: identify missing details, risky assumptions, dependencies and likely implementation blockers.
2. Propose a phased MVP that delivers one end-to-end vertical slice before adding all telemetry sources.
3. Recommend the simplest viable architecture and repository structure; avoid premature microservices.
4. Define a versioned canonical event JSON schema, baseline representation, classifier interface and audit record.
5. Give exact implementation tasks in dependency order, with acceptance criteria and test strategy for each phase.
6. Recommend a reproducible local lab using kind or Minikube and safe benign/suspicious test scenarios.
7. Plan fair evaluation against a rule-based baseline and a classical ML baseline using a held-out dataset. Include macro-F1, per-class precision/recall, false-positive rate, calibration, latency, cost and pre-filter missed-event rate.
8. Design safe failure behaviour for missing telemetry, stale baseline, classifier timeout/invalid output, and response failure.
9. Make Jev optional behind an adapter. Provide a fallback plan if access is unavailable.
10. Identify which parts of the original report need correction or qualification after implementation and measurements exist.

HOW TO WORK WITH ME
- First produce the review, architecture recommendation, phased plan, unresolved decisions and acceptance criteria. Do not

start by generating the entire codebase.
- Prioritize a working, testable MVP over tool count.
- Explain technical choices in straightforward language and give commands/configs only when they are actionable and safe for an isolated lab.
- Keep all attack simulation within the local lab; do not give the system broad production privileges.
- When implementation begins, work phase by phase, run or specify tests, and report files changed, commands run, results and remaining blockers.
- Never claim a test passed unless it actually ran and passed.
- Clearly distinguish measured results from illustrative examples.
- Ask me only for decisions that materially block the next step; otherwise state reasonable assumptions and proceed.

First deliver: (A) project critique, (B) MVP architecture, (C) ordered implementation roadmap, (D) repository structure, (E) event schema proposal, (F) evaluation plan, and (G) a short list of critical decisions/questions.

15. Quick Reference

Question	Answer
What are we building?	A contextual Kubernetes runtime threat detection and response pipeline.
What is the core problem?	Static image scans do not explain all runtime and orchestration threats; signals need context.
What is the first milestone?	A reproducible local cluster where one runtime event flows through normalization, classification, policy and audit.
What is Jev?	A proposed replaceable typed-decision classifier; access and behaviour must be verified.
What should be automated first?	Logging and alerts; human review for uncertain cases.
What should not be automated initially?	Pod termination; avoid disruptive response before evaluation.
How will success be measured?	Fair held-out comparison, per-class metrics, false-positive rate, calibration, latency and cost.
What is the main project contribution?	A tested integration of multi-source telemetry, workload context, classification, safe policy and evaluation.

Source basis: the supplied report, “Intelligent Security for Containers: ML-Based Runtime and Orchestration Threat Classification Using the Jev Model” (17 pages), and the user-provided architecture/workflow diagram. This brief includes planning recommendations and a Claude prompt in addition to report-derived details.